INTERVIEW GUIDE

NutritionAIagents

A stateful meal-planner backend built with Express, LangChain, and LangGraph, designed around a supervisor-driven conversational workflow.

CORE PATTERN
Supervisor / Router Graph

MAIN OUTCOME
Multi-turn meal planning with persisted session state

BEST INTERVIEW ANGLE
Stateful orchestration, not just prompt engineering

What This System Does

1. Collects profile information
2. Collects health measurements and derives metrics
3. Collects location and food preferences
4. Generates a localized weekly meal plan
5. Explains the plan and supports follow-up questions

> The important engineering problem is not meal generation alone. The real challenge is controlled multi-turn progression with resumable server-owned state.

Tech Stack

NODE.JS · EXPRESS 5 · LANGCHAIN · LANGGRAPH · OPENAI

Session storage is currently an in-memory `Map`. This is good for prototyping and interview demos, but not for production persistence.

Prepared as an interview document: architecture first, prompt details second.

System Design

The backend is organized as an API-driven workflow executor. Each user message becomes one graph invocation. The controller loads the saved session state, invokes the workflow, stores the new state, and returns only the latest assistant response.

API Layer

- `POST /api/agent/message`
- `POST /api/agent/reset`
- `POST /api/agent/extension`

Workflow Layer

- `session_router`
- `profile_collector`
- `health_analyser`
- `preference_detector`
- `plan_generator`
- `plan_explainer`

State Layer

- `messages`
- `profile`
- `health`
- `preferences`
- `mealPlan`
- `currentNode`
- `nodeComplete`

Request Lifecycle

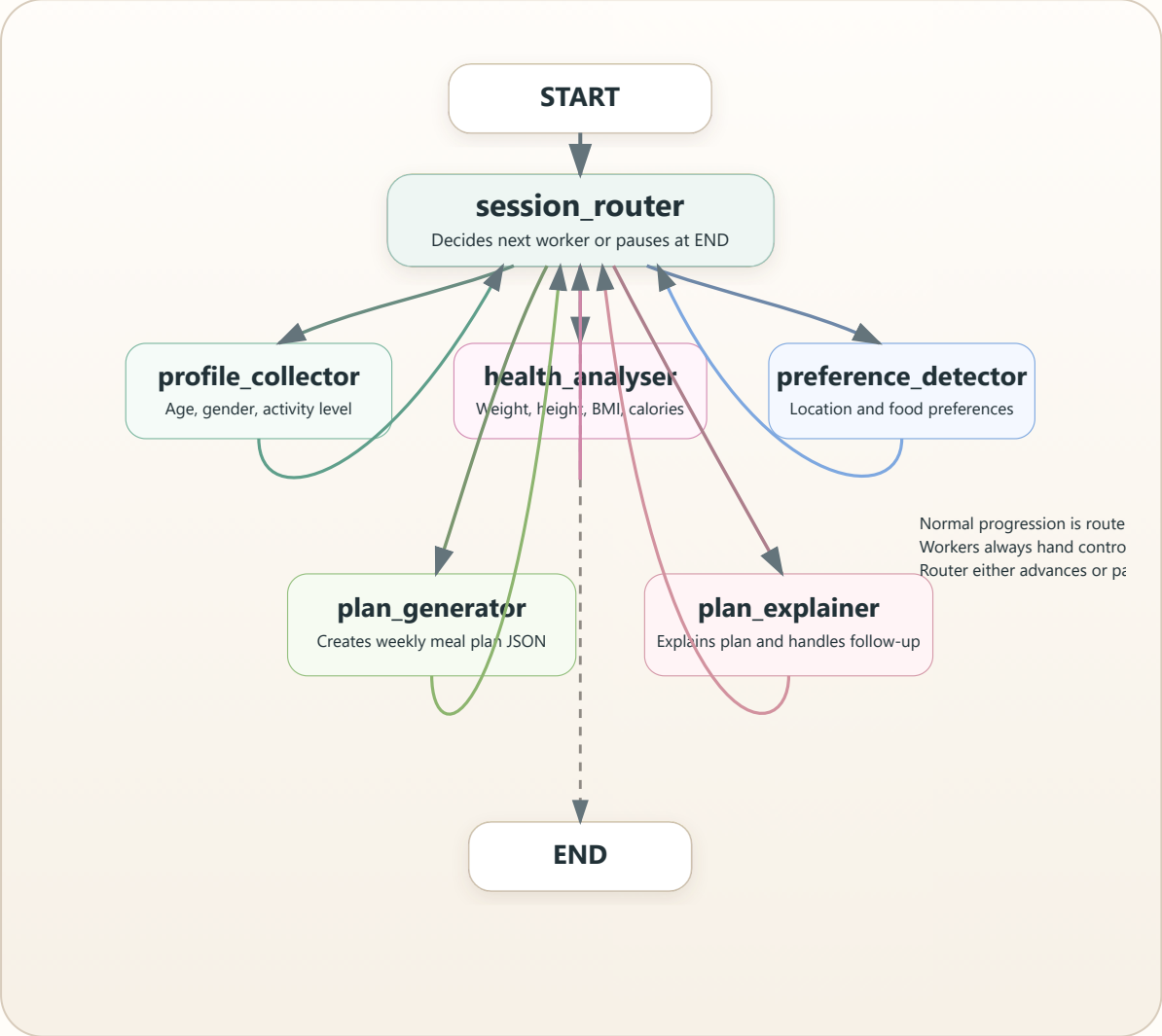
```
Client -> Controller -> Session Store (load)
       -> LangGraph agent.invoke(state)
       -> Session Store (save)
       -> Minimal JSON response back to client
```

Why this architecture is good

- The client is stateless; the server owns the workflow truth.
- Each node owns only one state slice, reducing accidental coupling.
- The router is the only orchestration point, so progression stays explainable.
- The design is easy to productionize by replacing the in-memory store with Redis or a database.

Realistic Flow Diagram

This diagram shows the actual supervisor pattern used by the project. Workers do their own job, then hand control back to the router. The router alone decides whether to pause, continue, or resume the explainer for post-plan questions.



> Interview line: “I intentionally moved to a supervisor graph so worker nodes do not decide orchestration directly. That makes the progression auditable, resumable, and easier to harden for production.”

Design Patterns and Why They Matter

Supervisor Pattern

The router is the single decision-maker for progression and pause behavior.

State Ownership

Each worker owns one state slice, which keeps responsibilities narrow and predictable.

Step Executor

Each HTTP request runs one graph invocation using previously persisted session state.

Why LangGraph instead of one big prompt?

- The conversation is multi-turn and structured.
- Different stages need different prompts, outputs, and validation logic.
- The system must resume from intermediate steps after every user turn.
- The final explainer needs access to saved plan data and session history.

Why combine heuristics with LLM parsing?

- Replies like `55`, `170 cm`, or `none` are better handled deterministically.
- Heuristics reduce avoidable model dependence for simple extraction.
- The LLM still handles broader natural-language variations when the input is less structured.

Why keep the client stateless?

- The server remains the source of truth for workflow state.
- It reduces client-side state corruption risk.
- It makes session recovery and multi-turn orchestration easier to control.

Node Responsibilities

The project is easiest to explain when you present it as five focused workers plus one router.

1. `profile_collector`

Collects age, gender, and activity level. If something is missing, it asks the next question. If complete, it marks the stage complete and returns control to the router.

2. `health_analyser`

Collects weight and height, then computes BMI, BMR, daily calories, and BMI category. Uses numeric heuristics plus LLM extraction.

3. `preference_detector`

Collects country, state, diet style, allergies, cuisine preference, and disliked foods. Handles replies like `none` reliably through local parsing.

4. `plan_generator`

Generates a seven-day meal plan with per-day nutrition totals. It uses the health and preference state to localize meals and respect restrictions.

5. `plan_explainer`

Explains the generated plan in simple, polite language with an Indian motivational tone. It stays active for follow-up questions once the plan exists.

API and State

Primary Endpoints

- `POST /api/agent/message`
- `POST /api/agent/reset`
- `POST /api/agent/extension`

Minimal response philosophy

The controller returns only:

- `success`
- `response`
- `currentNode`
- `nodeComplete`
- `mealPlan` only when it exists

State Model

```
{
  messages,
  profile,
  health,
  preferences,
  mealPlan,
  currentNode,
  nodeComplete,
  userId,
  sessionId,
  isFirstSession
}
```

Current storage tradeoff

> Session state is stored in memory with a JavaScript `Map`. This is ideal for rapid prototyping, but it is not durable, not horizontally scalable, and not safe for production deployment across multiple instances.

Interview Talking Points

Strong summary answer

“This is a stateful conversational meal-planner backend. I used Express for the API, LangGraph for orchestration, and a supervisor-router design so each node stays focused on one responsibility. The backend owns the session state, which allows the workflow to resume cleanly across multiple requests. The agent progresses from profile to health to preferences, then generates a weekly meal plan and finally explains it in a conversational way.”

Production improvements you can mention

- Replace in-memory sessions with Redis or persistent storage.
- Add request validation and schema validation for model outputs.
- Add retries, timeout guards, and structured logging around LLM calls.
- Add test coverage for graph progression and node extraction behavior.
- Add observability for node latency, failure rates, and session lifecycle.

What makes this stronger than a normal chatbot demo?

- It demonstrates orchestration, not just prompting.
- It models state ownership and progression semantics.
- It shows practical engineering tradeoffs around resumability and control flow.
- It supports a real post-generation conversational stage.

Recommended speaking angle: architecture, state flow, and design tradeoffs first. Prompt wording should be a secondary discussion.